AI-Assisted Development Methodologies: A Comparative Analysis

Project: Daily Habits Tracker Web App

Prepared By: Shrinivas Nemagoudar

Date: April 30, 2026

1. Executive Summary

This report evaluates three distinct AI-assisted development methodologies used to build the identical React/Firebase web application (Daily Habits Tracker). The objective was to determine the most efficient, scalable, and maintainable workflow for Avantro Labs.

Based on the execution and debugging phases of all three approaches, **the PRD-driven "single prompt" method (Approach 1) is entirely insufficient for production.** It leads to brittle code and a frustrating iteration cycle. **My recommended approach is a hybrid approach:** leveraging the Architecture-First method (Approach 2) to build the core Minimum Viable Product (MVP), followed by the User-Story-Driven method (Approach 3) for ongoing feature iteration.

2. Methodology Analysis

Approach 1: PRD-Driven Development (Single Prompt)

- **Execution:** Fed a complete Product Requirements Document (PRD) to the AI to generate the entire application structure in one massive output.
- **Code Quality (3/5):** The AI tends to take shortcuts, mock backend logic, or write monolithic files. Code separation is poor.
- **Debugging Experience (Poor):** Highly difficult. Because the AI generated everything at once, the context window was severely bloated. Asking the AI to fix a bug in the database logic often resulted in it accidentally breaking or rewriting the UI components.
- **Verdict:** Good for rapid, throwaway "toy" prototypes to test an idea, but completely unviable for actual product development where continuous iteration is required.

Approach 2: Architecture-First Development (Modular)

- **Execution:** Utilized an Excalidraw visual architecture diagram. The AI was explicitly restricted to building the application one specific component or state module at a time.
- **Code Quality (4.5/5):** Excellent. By forcing the AI to focus on a single node in the component tree, the resulting code was highly modular, clean, and properly separated.
- **Debugging Experience (Excellent):** Extremely easy. Because the codebase was built module-by-module, any bugs (like Firestore permission issues or state mismatches) were isolated to a single, small file. The AI could fix issues without hallucinating changes elsewhere.

- **Verdict:** The absolute best approach for building the initial MVP architecture and establishing a clean, scalable codebase.

Approach 3: User-Story-Driven Development (Iterative)

- **Execution:** Built the application incrementally by feeding the AI one Agile user story at a time (e.g., Auth -> Habit Creation -> Analytics).
- **Code Quality (4.5/5):** Very high. The code evolved naturally based on user requirements. Acceptance criteria ensured no logical steps were skipped.
- **Debugging Experience (Great):** Very manageable. Testing was done against specific acceptance criteria after each story, meaning bugs were caught immediately before building the next feature on top of them.
- **Verdict:** The ideal approach for extending an application. Once the core architecture is set, user stories provide the perfect context size for an AI to safely add new features.

4. Final Recommendation:

Software development is inherently an iterative process; an app is never finished in one try, it must be tweaked until it feels "right." Therefore, relying on a single mega-prompt (Approach 1) is a flawed paradigm.

Moving forward, I can adopt the following standard operating procedure for AI-assisted development:

1. **Phase 1 (The MVP):** Utilize **Approach 2 (Architecture-First)**. Engineers must draft visual component trees and database schemas, prompting the AI to build the foundational scaffolding and core layout module-by-module. This ensures the skeleton of the app is robust.
2. **Phase 2 (Continuous Iteration):** Once the MVP is rendering successfully, pivot entirely to **Approach 3 (User-Story-Driven)**. Engineers will feed the AI prioritized user stories to build out the functional features, ensuring the app evolves logically according to user needs without breaking the architecture.